

Technical Assessment

Fullstack Developer Intern -Skynet Flight Operations Module

Company: AIRMAN Aeronautics Pvt. Ltd.

Role: Fullstack Developer Intern

Product: Skynet -Flight School / Aviation Academy Operations SaaS

Assessment Duration: 24 hours

Submission Format: GitHub repository + README + screenshots / screen recording

Difficulty Level: Advanced Internship Assessment

Objective: Evaluate whether the candidate can build a real fullstack workflow using AIRMAN's product context, not a generic CRUD application.

1. Assessment Context

AIRMAN is building intelligent technology for aviation training, flight operations, and cockpit intelligence.

Our ecosystem includes:

TOGA: AI-powered pilot learning, study, logbook, and pre-flight planning app

Skynet: flight school and aviation academy operations SaaS

XB70: next-generation pilot flight computer and cockpit intelligence device

Sierra AI: AI-powered maintenance and MRO workflow intelligence

For this assessment, focus on **Skynet**.

Skynet is used by Flight Training Organisations to manage training operations, aircraft readiness, instructors, cadets, dispatch, compliance, and training progress.

Your task is to build a **mini fullstack Skynet Flight Dispatch + Training Progress module**.

This is not a generic dashboard or CRUD app. The workflow must reflect real aviation operations where role permissions, aircraft readiness, training progress, and audit trails matter.

2. Tech Stack

Use the following stack:

Backend

FastAPI

Python

PostgreSQL

- SQLAlchemy or SQLModel preferred
- Alembic migrations preferred
- Pydantic models
- REST APIs

Frontend

React

TypeScript

Tailwind CSS

React Query / TanStack Query preferred

React Router preferred

Optional

- Docker Compose

- JWT authentication
- Pytest
- Playwright
- GitHub Actions

3. Core Product Scenario

A flight school uses Skynet to manage daily sorties.

The system must support:

1. Cadets
2. Instructors
3. Aircraft
4. Bases
5. Sorties
6. Aircraft readiness
7. Dispatch workflow
8. Training progress update
9. Role-based access
10. Audit logs

The workflow:

```text

Sortie is scheduled

→ Dispatcher releases sortie

→ Sortie becomes airborne

→ Sortie lands

→ Instructor submits training progress

→ CFI approves training progress

→ Sortie is closed

```

Exception workflow:

```text

Aircraft lands

→ Defect is reported

→ Aircraft becomes grounded

→ Sortie cannot close until recovery decision is added

```

4. Required User Roles

Implement at least these roles:

```text

ADMIN

DISPATCHER

INSTRUCTOR

CFI

CADET

MAINTENANCE\_OFFICER

```

Minimum role expectations:

Role	Permissio
------	-----------

ADMIN	Can view and manage all records
DISPATCHER	Can create, release, mark airborne, mark landed
INSTRUCTOR	Can submit training progress
CFI	Can approve training progress
CADET	Can view own sortie and progress only
MAINTENANCE_OFFICER	Can mark aircraft grounded and add defect note

You may mock authentication if needed, but role handling must exist.

5. Required Data Model

Design PostgreSQL tables for at least:

users

```text

id

full\_name

email

role

base\_id

created\_at

```

Bases

```text

id

name

code

location

created\_at

```

aircraft

```text

id

registration

aircraft\_type

base\_id

status

tbo\_remaining\_hours

created\_at

```

Aircraft status:

```text

READY

SCHEDULED

AIRBORNE

LANDED

GROUNDED

MAINTENANCE

```

sorties

```text

id  
sortie\_number  
cadet\_id  
instructor\_id  
aircraft\_id  
base\_id  
lesson\_type  
scheduled\_start  
scheduled\_end  
actual\_start  
actual\_end  
status  
delay\_minutes  
created\_at  
updated\_at  
...

Sortie status:

```text

SCHEDULED

RELEASED

AIRBORNE

LANDED

TRAINING_SUBMITTED

TRAINING_APPROVED

CLOSED

CANCELLED

AIRCRAFT_GROUNDED

RECOVERY_REQUIRED

...

training_progress

```text

id

sortie\_id

cadet\_id

instructor\_id

lesson\_type

maneuver\_score

communication\_score

situational\_awareness\_score

remarks

status

submitted\_at

approved\_by

approved\_at

...

Training status:

```text

DRAFT

SUBMITTED

APPROVED

REJECTED

```

defects

```text

id

aircraft_id

sortie_id

reported_by

severity

description

status

created_at

```

Defect severity:

```text

LOW

MEDIUM

HIGH

CRITICAL

```

audit\_logs

```text

id
actor_id
action
entity_type
entity_id
old_value
new_value
created_at
` ` `

6. Required Backend APIs

Build REST APIs for the following.

Auth / Mock User

` ` ` http
POST /auth/login
GET /auth/me
` ` `

You may implement simple mock login using email + role.

Users

` ` ` http

GET /users

GET /users/{id}

```

---

Aircraft

```http

GET /aircraft

GET /aircraft/{id}

PATCH /aircraft/{id}/ground

PATCH /aircraft/{id}/ready

```

---

Sorties

```http

POST /sorties

GET /sorties

GET /sorties/{id}

PATCH /sorties/{id}/release

PATCH /sorties/{id}/airborne

PATCH /sorties/{id}/landed

PATCH /sorties/{id}/cancel

PATCH /sorties/{id}/close

```

---

## Training Progress

```http

POST /training-progress

GET /training-progress/{sortie_id}

PATCH /training-progress/{id}/submit

PATCH /training-progress/{id}/approve

PATCH /training-progress/{id}/reject

```

---

## Defects

```http

POST /defects

GET /defects

GET /defects/{id}

PATCH /defects/{id}/resolve

```

---

## Audit Logs

```http

GET /audit-logs

```

---

## 7. Required Backend Rules

Your backend must enforce business rules.

Sortie State Rules

Valid state transitions:

```text

SCHEDULED → RELEASED

RELEASED → AIRBORNE

AIRBORNE → LANDED

LANDED → TRAINING_SUBMITTED

TRAINING_SUBMITTED → TRAINING_APPROVED

TRAINING_APPROVED → CLOSED

```

Invalid transitions must be rejected.

Examples:

```text

SCHEDULED → AIRBORNE = invalid

AIRBORNE → RELEASED = invalid

LANDED → CLOSED without training approval = invalid

CLOSED → RELEASED = invalid

` ` `

Aircraft Rules

- Grounded aircraft cannot be assigned to a new sortie.
- Grounded aircraft cannot be released.
- Aircraft becomes `AIRBORNE` when sortie is airborne.
- Aircraft becomes `LANDED` when sortie lands.
- If a defect is reported after landing, aircraft becomes `GROUNDED`.
- Aircraft can become `READY` only after defect is resolved.

Training Progress Rules

- Only assigned instructor can submit training progress.
- CFI can approve or reject training progress.
- Cadet cannot edit training progress.
- Sortie cannot close before CFI approval.
- Scores must be between 1 and 5.
- Remarks cannot be empty during submission.

RBAC Rules

- Cadet can view only own sortie and progress.
- Instructor can view assigned sorties.
- Dispatcher can manage sortie state.
- Maintenance officer can ground aircraft.
- CFI can approve training progress.
- Admin can access all.

Audit Rules

Create audit logs for:

- Sortie creation
- Sortie release
- Airborne
- Landed
- Cancelled
- Closed
- Aircraft grounded
- Aircraft marked ready
- Training submitted
- Training approved/rejected
- Defect created/resolved

8. Required Frontend Screens

Build a React + TypeScript + Tailwind UI.

The UI does not need to be visually perfect, but it must be clean, usable, and structured.

Screen 1: Login / Role Selector

Allow user to login as:

- Admin
- Dispatcher
- Instructor
- CFI
- Cadet
- Maintenance Officer

You may use mock users.

Screen 2: Operations Dashboard

Show:

- Total sorties today
- Released sorties
- Airborne sorties
- Landed sorties
- Grounded aircraft
- Training approvals pending
- Delayed sorties

Screen 3: Sortie Board

Show sortable/filterable sortie list.

Each sortie card/table row should show:

- Sortie number
- Cadet
- Instructor
- Aircraft
- Base
- Lesson type
- Scheduled time
- Status badge
- Delay minutes
- Available actions based on role

Required actions:

- Release
- Mark Airborne
- Mark Landed
- Cancel

- Close

Only show actions allowed for the logged-in role.

Screen 4: Sortie Detail

Show:

- Sortie details
- Aircraft details
- Cadet and instructor details
- Current status
- Timeline of state changes
- Training progress status
- Defect status if any
- Audit history for this sortie

Screen 5: Training Progress Form

Instructor can submit:

- Maneuver score
- Communication score
- Situational awareness score
- Remarks

CFI can:

- View submitted progress
- Approve
- Reject with remarks

Cadet can:

- View approved progress only

Screen 6: Aircraft Readiness / Defect Panel

Maintenance officer can:

- View aircraft
- Report defect
- Mark aircraft grounded
- Resolve defect
- Mark aircraft ready

Show:

- Aircraft registration
- Type
- Base
- Status
- TBO remaining
- Defect count
- Active defect severity

Screen 7: Audit Log View

Show:

- Actor
- Action
- Entity
- Timestamp
- Old value
- New value

Allow filter by:

- Entity type
- Actor role
- Action

9. Required Frontend Behaviors

Your frontend must handle:

- Loading states
- Empty states
- Error states
- Form validation
- Disabled buttons during API calls
- Success/error toast or message
- Role-based UI action visibility
- API error display
- Status badges
- Basic responsive layout
- Data refetch after mutation

10. Required Seed Data

Provide seed data for:

- 2 bases
- 3 aircraft
- 6 users across roles
- 5 sorties with different statuses
- 2 training progress records
- 2 defects
- Audit logs if generated through seed or actions

Example users:

```text

Admin User -ADMIN

Dispatch Officer -DISPATCHER

Capt. Rao -INSTRUCTOR

Chief Flying Instructor -CFI

Arjun Menon -CADET

Maintenance Officer -MAINTENANCE\_OFFICER

````

Example aircraft:

```text

VT-ABC -Cessna 172 -READY

VT-SKY -Piper PA-28 -GROUNDED

VT-AIR -Diamond DA40 -READY

````

11. Required Testing

Include basic tests.

Backend Tests

Use Pytest.

Minimum tests:

1. Dispatcher can release scheduled sortie.
2. Cadet cannot release sortie.
3. Cannot mark scheduled sortie airborne directly.
4. Grounded aircraft cannot be released.
5. Instructor can submit training progress.

6. Cadet cannot submit training progress.
7. CFI can approve training progress.
8. Sortie cannot close before CFI approval.
9. Aircraft becomes grounded when defect is reported.
10. Audit log is created for sortie release.

Frontend Tests -Bonus

Playwright or Cypress tests are optional but preferred.

Suggested E2E:

```
```text
```

Login as dispatcher

→ open sortie board

→ release sortie

→ mark airborne

→ mark landed

→ verify status changes

```
```
```

12. Bonus Features

Bonus points for:

- Docker Compose for frontend, backend, database
- Alembic migrations
- JWT authentication
- Refresh token handling

- React Query implementation
- Role-based route guards
- Pagination and filters
- Search in sortie board
- Audit timeline
- Playwright E2E tests
- Backend test coverage report
- GitHub Actions CI
- OpenAPI schema clarity
- Proper error response format
- Optimistic UI with rollback
- WebSocket or polling for live sortie status
- Dark mode

13. Required Folder Structure

Recommended structure:

```
```text
```

```
airman-fullstack-assessment/
```

```
 backend/
```

```
 app/
```

```
 main.py
```

```
 core/
```

```
 config.py
```

```
 security.py
```

```
 db/
```

```
 database.py
```

```
 models.py
```

```
 migrations/
```

```
 schemas/
```

user.py

aircraft.py

sortie.py

training\_progress.py

defect.py

audit\_log.py

api/

routes/

auth.py

users.py

aircraft.py

sorties.py

training\_progress.py

defects.py

audit\_logs.py

services/

sortie\_service.py

training\_service.py

aircraft\_service.py

audit\_service.py

tests/

test\_sorties.py

test\_rbac.py

test\_training\_progress.py

requirements.txt

README.md

frontend/

src/

api/

client.ts

sorties.ts

aircraft.ts

trainingProgress.ts

auditLogs.ts

components/

StatusBadge.tsx

RoleGuard.tsx

SortieCard.tsx

AuditTimeline.tsx

pages/

Login.tsx

Dashboard.tsx

SortieBoard.tsx

SortieDetail.tsx

TrainingProgress.tsx

AircraftReadiness.tsx

AuditLogs.tsx

hooks/

useAuth.ts

useSorties.ts

useAircraft.ts

types/

sortie.ts

aircraft.ts

user.ts



trainingProgress.ts

App.tsx

main.tsx

package.json

README.md

docs/

architecture.md

api-contract.md

ai-usage-disclosure.md

known-limitations.md

README.md

...

---

## 14. README Requirements

Your main README must include:

1. Project overview
2. Tech stack used
3. Setup instructions
4. Environment variable instructions
5. How to run backend
6. How to run frontend
7. How to run migrations / seed data
8. How to run tests
9. API summary

10. User roles and permissions
11. Business rules implemented
12. Known limitations
13. What you would improve with more time
14. AI usage disclosure

---

## 15. AI Usage Disclosure

AI tools are allowed, but usage must be disclosed.

Create:

```
```text
```

```
docs/ai-usage-disclosure.md
```

```
```
```

Answer:

1. Did you use AI tools? If yes, where?
2. What did AI generate?
3. What did you manually review or change?
4. Which AI-generated suggestion did you reject and why?
5. Which part of the project did you personally design?
6. Which part are you least confident about?
7. Pick one backend function and explain it line by line.
8. Pick one frontend component and explain it line by line.

During interview, you may be asked to modify/debug your own code without AI assistance.

---

## 16. Evaluation Rubric

| Area                                     | Weight      |
|------------------------------------------|-------------|
| Backend architecture and API quality     | 18%         |
| Database design and business rules       | 15%         |
| Frontend implementation and UX           | 15%         |
| Fullstack integration                    | 12%         |
| RBAC and permission enforcement          | 12%         |
| State transition correctness             | 10%         |
| Audit logging and data integrity         | 6%          |
| Testing quality                          | 5%          |
| Code quality and maintainability         | 4%          |
| README, documentation, and AI disclosure | 3%          |
| <b>Total</b>                             | <b>100%</b> |

---

## 17. Strong Candidate Signals

A strong candidate will:

- Build a working end-to-end module
- Use clean FastAPI structure
- Design meaningful PostgreSQL models
- Enforce business rules on backend, not only frontend
- Use clear Pydantic schemas
- Handle role permissions correctly
- Implement valid/invalid state transitions
- Build a usable React + TypeScript UI
- Handle loading, error, and empty states
- Add useful audit logs
- Write at least basic backend tests
- Explain trade-offs clearly
- Use AI responsibly but understand the code

---

## 18. Red Flags

Reject or reconsider candidates who:

- Build only frontend with fake static data
- Build only backend without usable UI
- Put all backend code in one file
- Ignore role permissions
- Enforce security only by hiding frontend buttons
- Allow invalid sortie transitions
- Allow grounded aircraft to be dispatched
- Allow cadets to edit training progress
- Do not create audit logs
- Do not provide setup instructions
- Cannot run their own project
- Cannot explain their code
- Use AI-generated code without understanding it
- Do not include tests
- Hardcode everything without reusable structure

---

## 19. Live Review Requirement

After submission, there may be a 30-minute live technical review.

Candidate may be asked to perform tasks without AI assistance:

1. Add a new invalid state transition test.
2. Add a new role permission rule.
3. Debug a failing API endpoint.
4. Modify one React component.
5. Add one field to the sortie model.
6. Explain how audit logs are created.
7. Explain why backend permissions are more important than frontend button hiding.
8. Explain one FastAPI route line by line.

9. Explain one SQL model relationship.

10. Explain one React Query mutation flow.

This live review is part of the assessment.

---

20. Final Candidate Note

AIRMAN is building aviation technology. In aviation workflows, correctness, permissions, state transitions, and auditability matter.

We are not evaluating whether you can build a generic CRUD dashboard. We are evaluating whether you can build a reliable fullstack workflow where:

- The right person performs the right action
- Aircraft readiness is respected
- Sortie state transitions are valid
- Cadet training progress is protected
- Critical actions are auditable
- Frontend and backend stay consistent
- Errors are handled clearly
- Code is maintainable

Build this as if it is the foundation of a real Skynet module that aviation academies may use in production.